

Anatomia da Programação Semanal

O mesmo sistema explicado quatro vezes, cada vez com um passo a mais de detalhe. Pare no nível que responde à sua pergunta.

Modelo C4 (Simon Brown) Diagramas 8, desenhados no Design System Levantado do código em 28/08/2026

Fonte de verdade docs/ENTENDA-O-SISTEMA.md

ORIENTAÇÃO

Como ler os diagramas

Estes desenhos seguem o **modelo C4** — uma convenção para desenhar software do jeito que se usa um mapa: você dá zoom até enxergar o que precisa, e não mais que isso.

NÍVEL	NOME TÉCNICO	A ANALOGIA	RESPONDE
1	Contexto	O mapa do país	Com quem esse sistema conversa?
2	Contêineres	O mapa da cidade	De que peças grandes ele é feito?
3	Componentes	A planta do prédio	O que existe dentro de uma das peças?
4	Implantação	A foto de satélite	Onde isso tudo roda de verdade?

O que cada cor significa

**Pessoa da obra**
Quem usa o sistema por dentro da operação.

**Pessoa de fora**
Não usa a tela — consome os números.

**Peça nossa**
Código que a Timenow escreve e mantém.

**Onde o dado mora**
Arquivo ou base. Guarda estado.

**Sistema de terceiros**
Existe, funciona, não é nosso.

**Pronto e desligado**
O código existe. Não está em uso.

Uma regra que vale para tudo aqui: a seta significa “chama”, não “depende de”. Se A aponta para B, é A quem toma a iniciativa. E toda seta tem um verbo em cima dela — seta sem rótulo é só “relacionado de algum jeito”, o que não informa nada.

O PROBLEMA

Em uma frase

Uma obra tem várias empresas contratadas. Toda segunda-feira cada uma promete o quanto vai produzir de cada serviço, dia a dia. No fim da semana, diz o quanto produziu de verdade. **Este sistema é o lugar onde essa promessa é feita, conferida, aprovada e medida.**

O nome técnico da prática é *Last Planner System*. O indicador principal chama-se **PPC** — Percentual do Plano Concluído. Traduzindo: *de tudo que essa atividade prometeu para a semana, quanto saiu?*

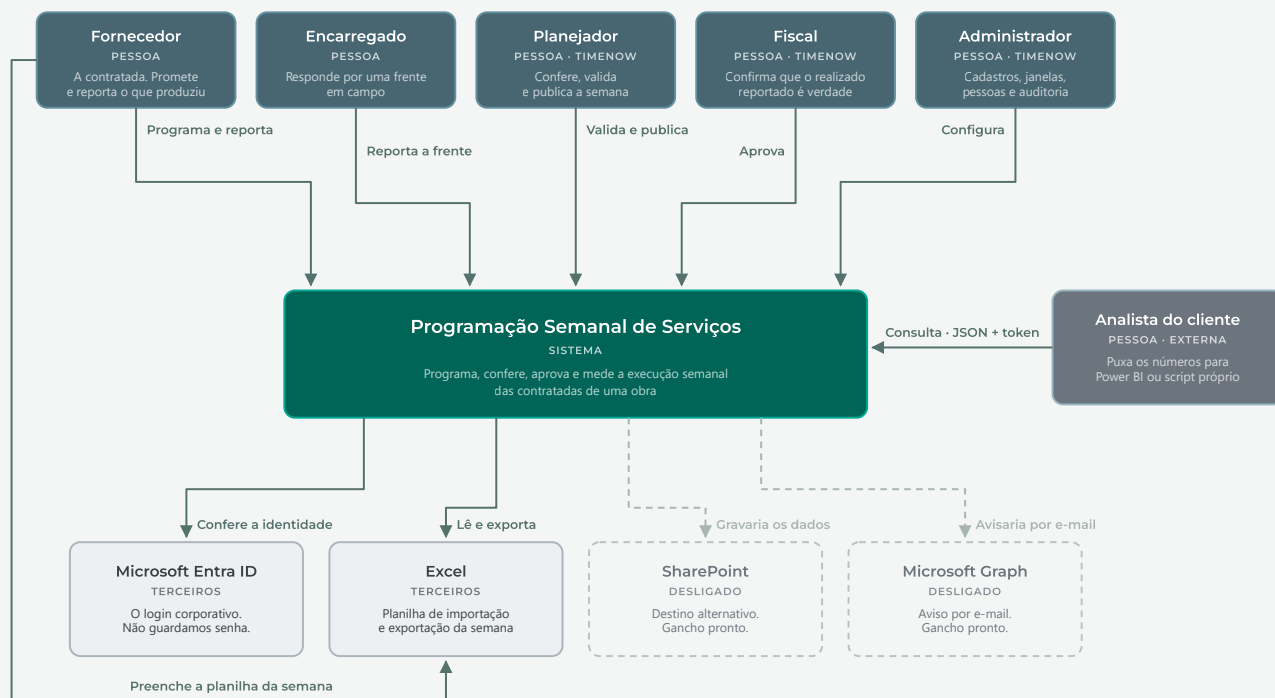
Cuidado com uma palavra. Neste projeto, *ambiente* significa a caixa de dados de um cliente — nunca “ambiente de desenvolvimento/produção”. Para isso o projeto usa **implantação**.

NÍVEL 1

Contexto: com quem o sistema conversa

Cinco papéis alimentam o sistema por dentro. Um sexto, do lado do cliente, entra por outra porta. E quatro sistemas de terceiros ficam nas bordas — dois deles ainda desligados.

FIGURA 1 Contexto do sistema



As duas caixas tracejadas são a informação mais honesta do desenho: o código existe, está escrito, e não está ligado.

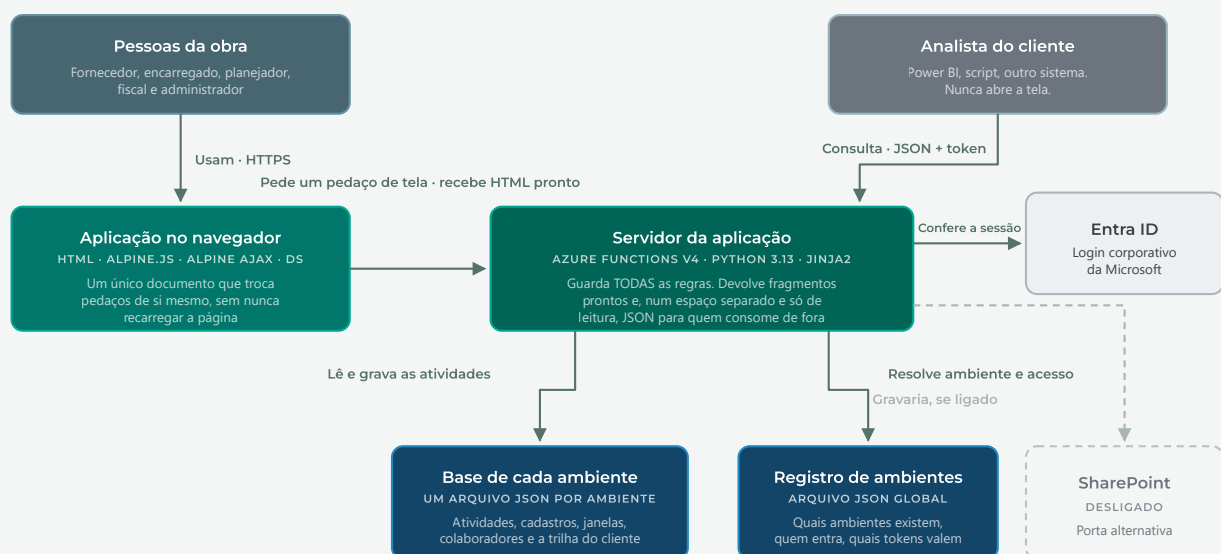
Por que Excel aparece num diagrama de arquitetura. O Excel roda em planilha. Tratar isso como um problema a ser eliminado seria arrogância de software — então a planilha entrou como caminho de primeira classe, com modelo gerado e conferência linha a linha.

NÍVEL 2

Contêineres: de que peças o sistema é feito

Zoom na caixa verde do meio. São quatro peças, e a mais importante do desenho é a seta do meio: o servidor devolve **tela pronta**, não dado cru.

FIGURA 2 Contêineres



O navegador nunca fala com a base. Nenhuma seta liga a aplicação a um cilindro — todo acesso a dado passa pelo servidor.

Três decisões que explicam o desenho inteiro

1. **O servidor devolve tela pronta, não dado cru.** No jeito mais comum de fazer software web hoje, o servidor manda dados e o navegador monta a tela. Aqui é o contrário. *Por que isso importa para quem não programa:* a tela não pode discordar do servidor, porque ela não calcula nada. Não existe a situação clássica de "na minha tela aparece um número, no relatório aparece outro".
2. **Não existe etapa de compilação.** Sem bundler, sem build, sem CDN. Os arquivos que estão na pasta são exatamente os que rodam. Quem abrir o projeto daqui a três anos consegue ler o que está acontecendo.
3. **Existem duas portas de entrada, propositalmente diferentes.**

	PORTA DA APLICAÇÃO	PORTA DE LEITURA
Endereço	/api/	/api/dados/v1/
Quem usa	O navegador de quem trabalha na obra	Power BI, script, sistema do cliente
Credencial	A sessão do Azure	Um token emitido para aquele ambiente
Devolve	HTML pronto	JSON
Pode escrever?	Sim	Não. Só leitura, sempre

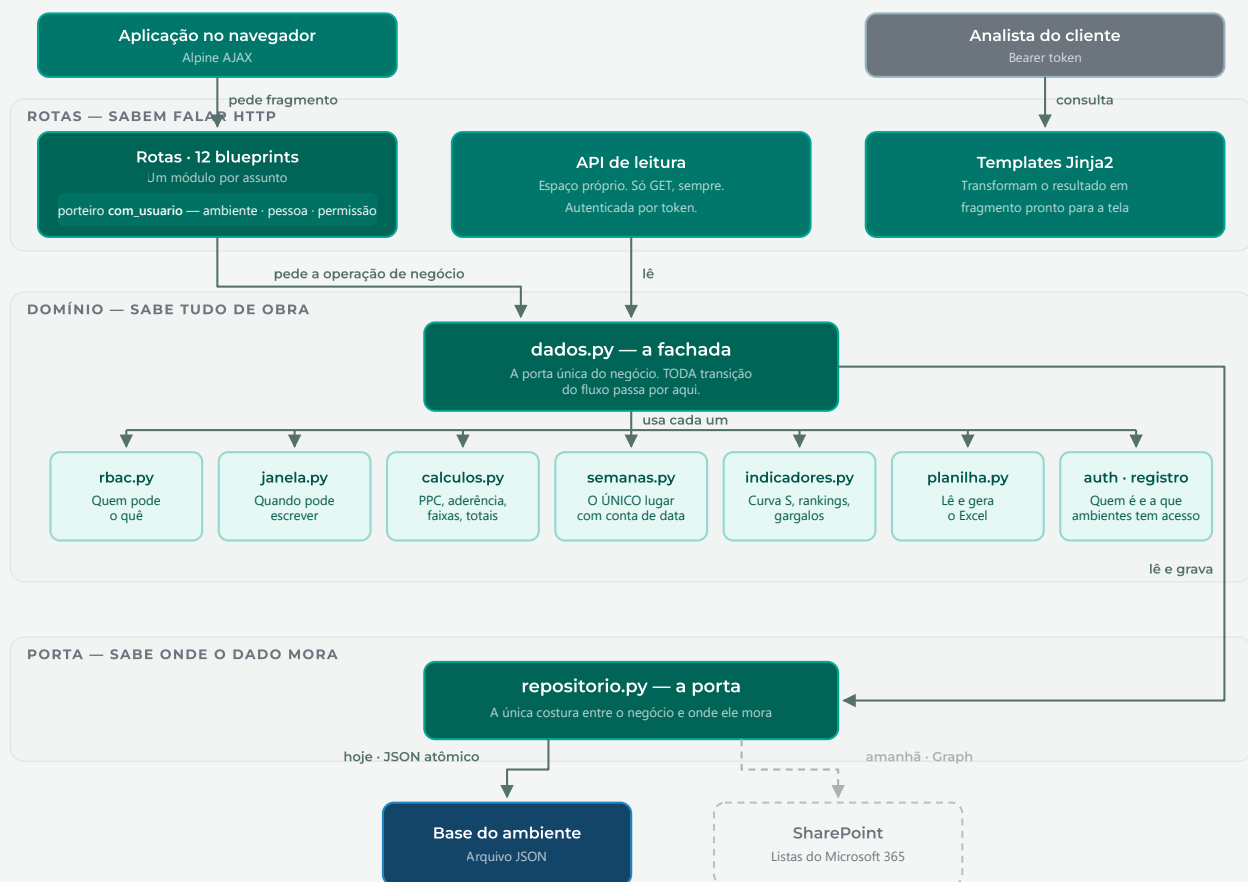
O token é quem resolve o ambiente. Não há cookie, não há cabeçalho de aba. Um token não alcança outro ambiente porque não existe caminho para isso — não porque uma verificação impede.

NÍVEL 3

Componentes: o que existe dentro do servidor

O servidor tem três camadas, e a regra entre elas é rígida: cada uma só conversa com a de baixo. As faixas do desenho são essas camadas.

FIGURA 3 Dentro do servidor, por camada



O `com_usuario` não é um módulo ao lado das rotas — é um decorador que embrulha cada uma delas. Por isso aparece dentro da caixa, não ao lado.

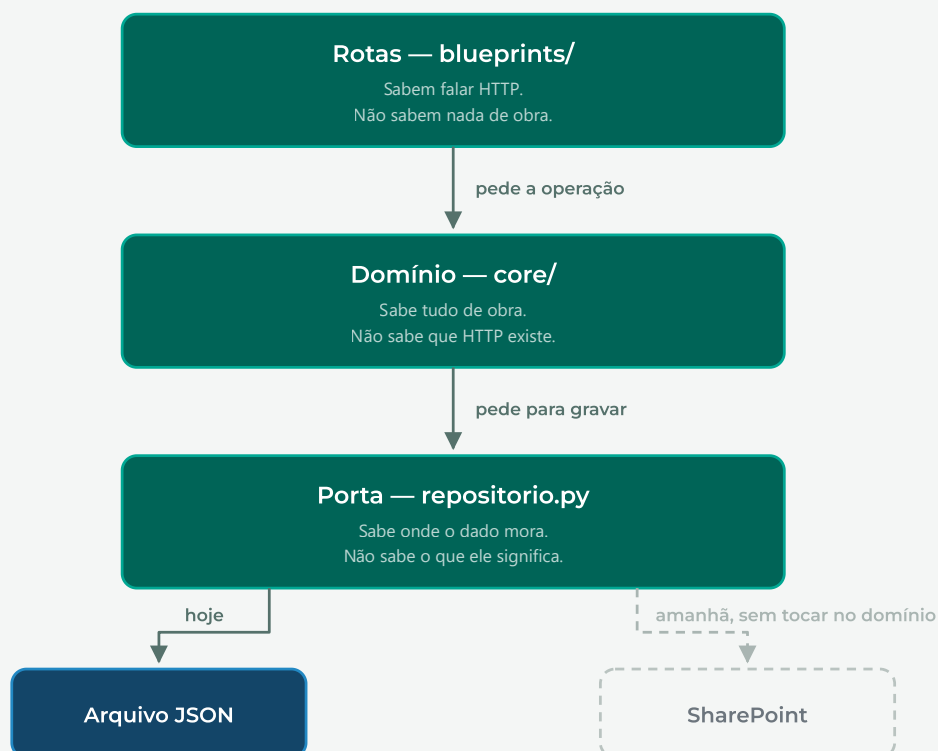
Componentes que merecem nome próprio

COMPONENTE	O QUE FAZ, E POR QUE É ASSIM
<code>dados.py</code> a fachada	Todo caminho passa por aqui. Nenhuma rota conversa com o arquivo direto. É o que garante que “publicar” signifique exatamente a mesma coisa vindo da tela, da planilha ou de um script.
<code>com_usuario</code> o porteiro	Um decorador no topo de cada rota. Faz sempre a mesma coisa, nesta ordem: 1) o pedido veio mesmo da aplicação? 2) de qual ambiente? 3) quem é a pessoa? 4) ela pode? Se a rota executa, as quatro respostas já são sim.
<code>semanas.py</code>	O único módulo autorizado a fazer aritmética de data. Semana ISO tem armadilhas — a semana 1 é a que contém a primeira quinta-feira do ano, e há anos com 53 semanas. Concentrar isso num arquivo é o que impede o clássico “o relatório de dezembro sumiu”.
<code>repositorio.py</code> a porta	Uma classe abstrata com cerca de 15 métodos. Trocar de JSON para SharePoint é escrever outra implementação dela.
<code>registro.py</code>	Vive <i>acima</i> dos ambientes. É dono do nome do projeto, do nome do cliente, de quem é membro e dos tokens de leitura. Tem trilha de auditoria própria.

A REGRA

Por que a ordem das camadas não é negociável

FIGURA 4 A dependência só desce

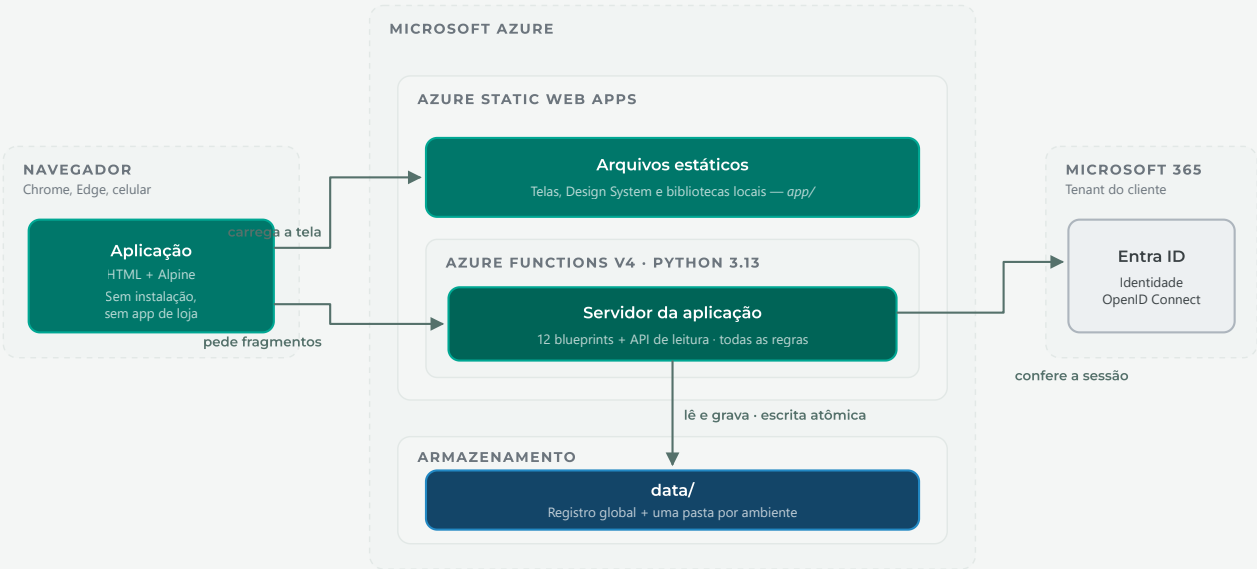


Trocar de JSON para SharePoint é o valor de uma variável de configuração — a regra de que a soma dos sete dias tem de bater com o total continua escrita num lugar só, intocada.

NÍVEL 4

Implantação: onde isso roda

FIGURA 5 Produção no Azure



Sem instalação e sem app de loja: quem trabalha na obra abre um endereço no celular ou no PC.

O mesmo sistema, na máquina de quem desenvolve

	EM PRODUÇÃO	NA MÁQUINA LOCAL
Quem serve a tela	Azure Static Web Apps	<code>swa start</code> , ou um servidor Python equivalente
Quem diz quem você é	Entra ID (login da empresa)	Modo demonstração — entra pelo cadastro, com seletor de perfil
Onde o dado mora	<code>data/</code> no Azure	<code>data/</code> na sua pasta
Como subir	Deploy do SWA	Dois cliques em <code>run.bat</code>

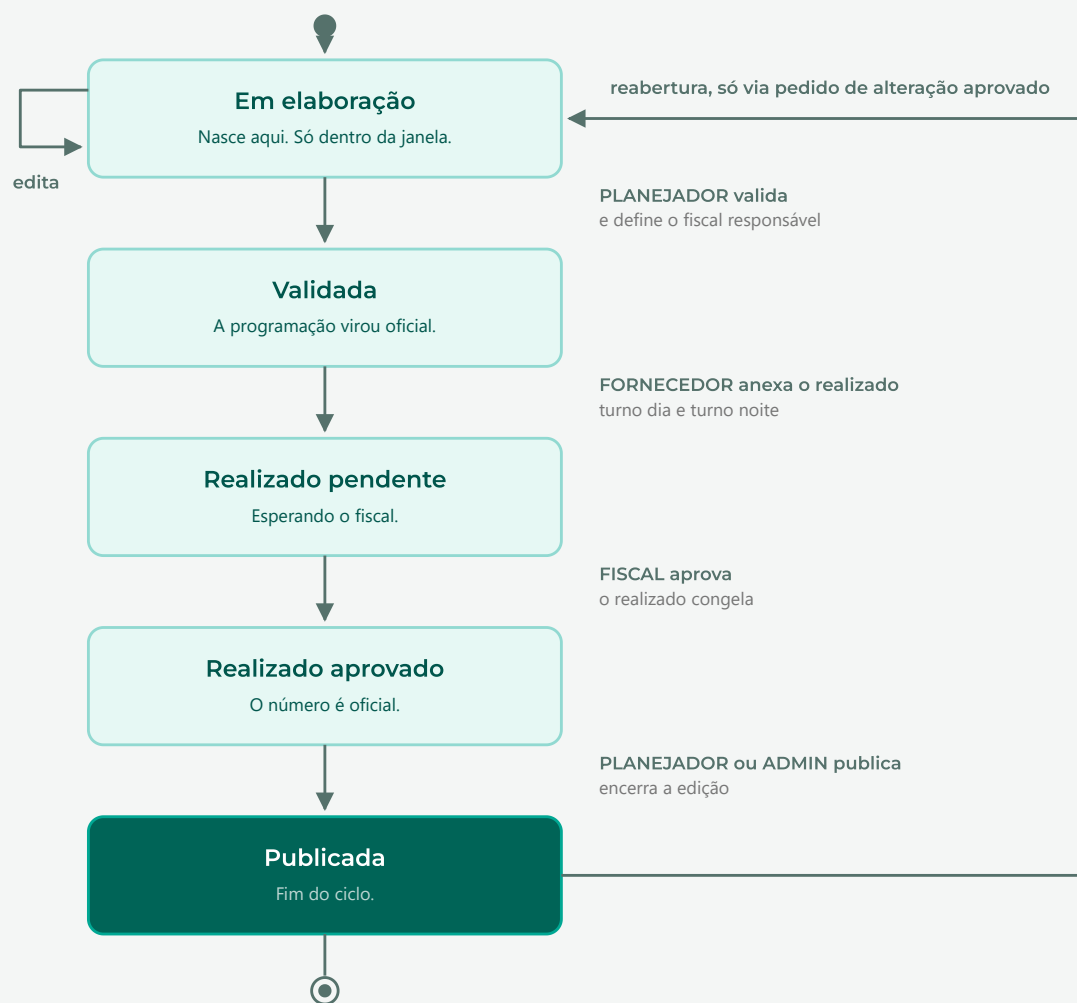
O modo demonstração **desliga sozinho** assim que o Azure passa a autenticar. Não é uma chave que alguém pode esquecer ligada: se existe sessão do Azure na requisição, o modo demo não roda.

Na primeira execução local o sistema cria duas obras fictícias — `demo-obra` e `demo-planta` — cada uma com seis semanas de histórico e duas à frente, ancoradas na semana corrente. Assim toda tela tem número e todo gráfico tem forma antes de alguém digitar qualquer coisa. Apagar `data/` regenera tudo.

OPERAÇÃO

O ciclo da semana

Uma atividade nasce, anda por cinco estados e termina. Cada passo tem um dono, e só ele move.

FIGURA 6 Os cinco estados e quem move cada um

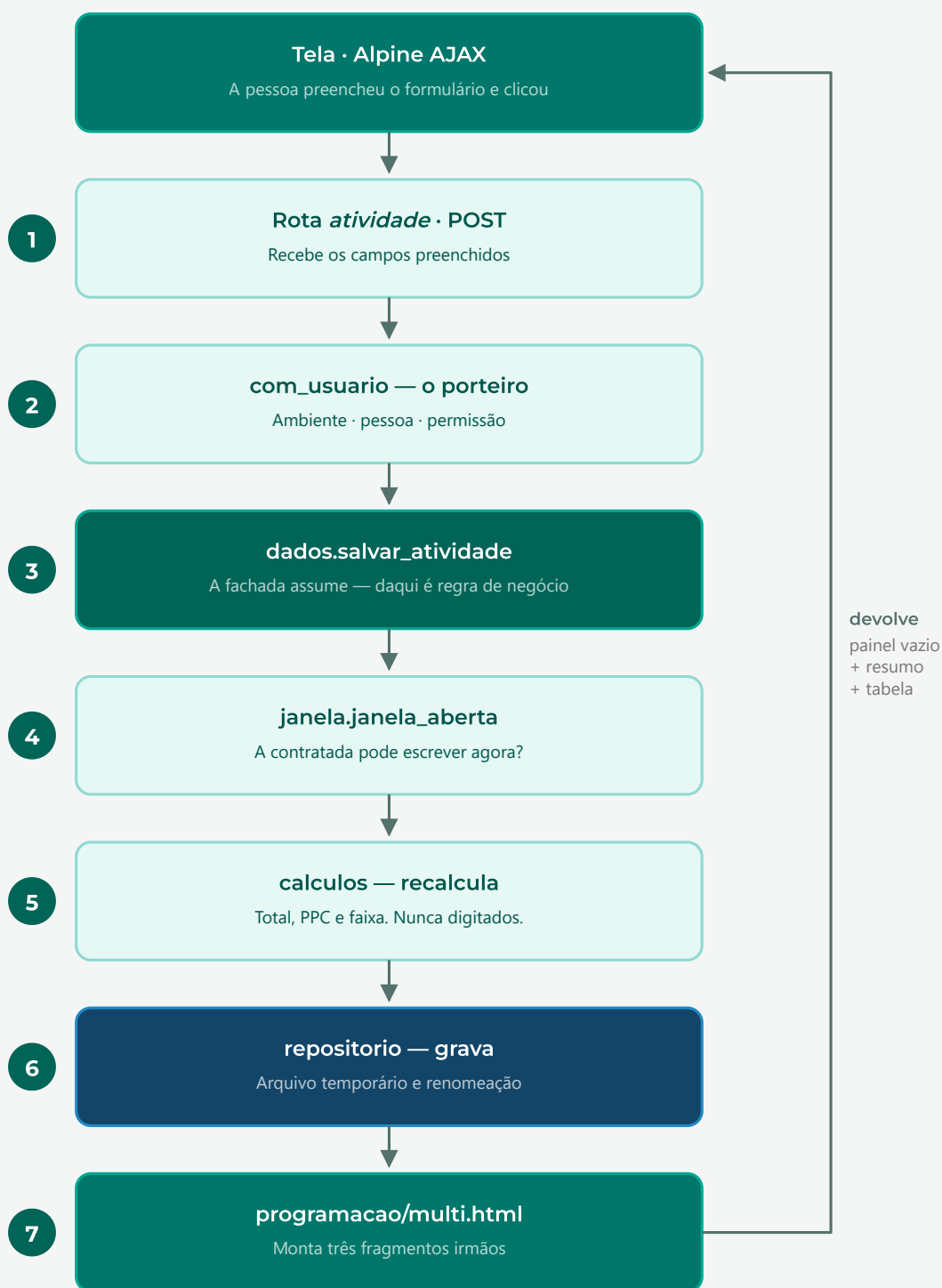
A única seta que sobe é a reabertura — e ela exige um pedido com motivo escrito, decidido por quem tem a permissão de aplicar.

Duas coisas diferentes que parecem uma só. *Situação* é onde a programação está no fluxo (em elaboração / validada / publicada). *Aprovação do realizado* é outra régua (pendente / aprovado). Uma atividade publicada pode ter realizado pendente — e isso não é um bug.

POR DENTRO

O caminho de um clique em “salvar”

FIGURA 7 Sete passos entre o clique e a tela atualizada



O painel volta **vazio** — é assim que ele fecha. Um clique, três partes da tela atualizadas, zero recarregamento.

Total, PPC e aderência são sempre calculados, nunca digitados. Não existe campo de total na tela. Não existe como salvar um PPC que não corresponda aos números.

OPERAÇÃO

A janela de programação

A regra que decide se uma contratada pode escrever *agora*. São três camadas de prioridade, e a ordem entre elas é o que faz a regra funcionar.

FIGURA 8 Três camadas, em ordem de prioridade



Fora da janela a contratada **NÃO** fica sem acesso — ela consulta normalmente. Só não escreve.

O detalhe que costuma passar despercebido: a liberação extraordinária vence a regra **nos dois sentidos**. Dentro do intervalo abre mesmo no dia errado; fora dele fecha mesmo que a semana estivesse liberada.

ACESSO

Quem pode o quê

Seis perfis. A tela esconde botão por conveniência; quem decide acesso é sempre o servidor.

PERFIL	VER	EDITAR	VALIDAR	REPORTAR	APROVAR	PUBLICAR	IMPORTAR	CONFIG.	AUDITORIA
Administrador Acesso total, mais pessoas e auditoria	✓	✓	✓	✓	✓	✓	✓	✓	✓
Planejador Valida, define o fiscal e publica	✓	✓	✓	—	—	✓	✓	✓	✓
Fiscal Aprova o realizado sob sua responsabilidade	✓	✓	—	—	✓	—	—	✓	—
Encarregado Responde pela frente em campo	✓	—	—	✓	—	—	—	—	—
Fornecedor Programa e reporta a própria empresa	✓	✓	—	✓	—	—	—	—	—
Visualizador Só leitura, com exportação	✓	—	—	—	—	—	—	—	—

Perfil é o que você pode fazer. Vínculo é o que você enxerga.

VÍNCULO	ENXERGA
timenow	Tudo, no ambiente ativo
fornecedor	Só a própria empresa, em toda tela, sempre
cliente	Tudo, no ambiente ativo

O recorte do fornecedor é aplicado **no servidor**, na fachada, antes de a tela existir. Não é um filtro de interface que alguém possa contornar digitando um endereço.

E, fora da tabela de propósito, o operador. É o papel global que administra o registro de ambientes, com três permissões que *nenhum perfil do cadastro possui*: gerir o registro, conceder acesso e gerir tokens. Ele vem da configuração da implantação, nunca do dado — porque o poder de enxergar todos os clientes não pode ser fabricado editando um registro.

OS INDICADORES

Como os números saem

Nenhum destes é digitado. Todos são calculados a cada leitura.

NÚMERO	A CONTA	O DETALHE QUE MUDA TUDO
Total previsto	Soma dos sete dias	Tem de bater com a produção prevista da semana, com tolerância de 0,5
Total realizado	Soma dos sete dias, turno dia + turno noite	—
PPC	realizado ÷ previsto, de uma atividade	É o indicador do Last Planner. É <i>por atividade</i> .
Aderência	soma do realizado ÷ soma do previsto de um conjunto	Não é a média dos PPCs. Aderência pondera pelo tamanho; a média de PPC, não.
Faixa	alta ≥ 80% · média 60–79% · baixa < 60%	Uma escala só — tabela, medidores, gráficos e planilha exportada
Avanço físico	Por atividade, como fração do escopo no horizonte, peso igual	Unidades da obra não são somáveis entre si

A diferença entre PPC médio e aderência é a que mais gera discussão em reunião. Se uma contratada tem uma atividade de 1000 m³ com 50% e outra de 10 m³ com 100%, o PPC médio dá 75% e a aderência dá cerca de 50%. **A aderência está certa** — foi ela que percebeu que o volume que importava não saiu.

Por que o avanço físico não é uma soma ingênua. As unidades da obra não são somáveis entre si — kg de armação com m³ de solo não dá número nenhum. O avanço é medido por atividade, como fração do escopo dela no horizonte, com peso igual entre atividades. Isso está escrito na legenda do próprio gráfico, na tela, para ninguém interpretar errado.

HONESTIDADE

O que está pronto e desligado

Três coisas existem no código e não estão em uso. Aparecem tracejadas nos diagramas.

O QUÊ	ESTADO	O QUE FALTA
SharePoint como base	O mapa de listas e colunas está escrito	Só a aquisição de token, isolada em dois métodos. Enquanto não existir, a integração recusa com uma mensagem dizendo o que configurar — em vez de falhar com erro 500 no meio da tela.
Aviso por e-mail	O envio pelo Microsoft Graph está escrito	Ninguém chama. Ficou fora porque <i>avisar automaticamente é decisão de processo, não de código</i> — e essa decisão ainda não foi tomada.
Modo demonstração	Ligado por padrão fora do Azure	Nada. Desliga sozinho quando há sessão do Azure.

Diagramas desenhados à mão em SVG, no Timenow Design System — mesma decisão do `ds/charts.js`, que desenha os gráficos do dashboard sem biblioteca para poder obedecer aos tokens. Paleta, tipografia e a regra de elevação por borda vêm de `app/ds/tokens.css`.

A fonte de verdade do conteúdo é `docs/ENTENDA-O-SISTEMA.md`, onde os mesmos oito diagramas existem em Mermaid para leitura no GitHub e no VS Code.